Referee Report, Round 3: "Honest Code Benchmark" Preregistration, Specification, and Code

What I audited: the round-3 copy of the author's tree at `/workspace/r3/tree` (commit `0754d6a` plus uncommitted and untracked changes). I diffed it against the round-2 copy at `/workspace/v2`. All builds and server runs used a scratch copy (`/workspace/scratch/r3/w`); nothing in the audited tree was changed. **Materials:** `preregistration.md`, `research-plan.md`, `task-specification.md`, `generate-data.js`, `data/sqlite/` (schema, build script, and manifest), `apps/` (both servers, `lib/sqlite.js`, `lib/sqlite_db.py`, the build script, and every arm), and `harness/`. **Review date:** Oct. 30, 2026, CT. "Verified" means I checked the claim by running code on the scratch copy, using Node 22.23.4, Python 3.13.5 (FastAPI 0.162.0), and headless Chrome 155. **Judged against these author decisions:** (a) Arms 7, 8, and 9 share one client, taken from Arm 1, and differ only in server code; the table sort runs on the client. (b) Arms 6, 10, and 11 read from SQLite with `LIMIT`/`OFFSET`. (c) The sort rule is ASCII case-insensitive, with ties broken by ascending `id`, booleans true-first, and filtering by `instr`. **Scope limit:** by the author's instruction, this audit measured no timings. It reports correctness only. Harness runs were used to see whether scripts complete; any timing output was discarded unread.

Verdict

Not ready to register. The SQLite work is a real step forward. Arms 10 and 11 now page from SQLite with `instr` and a whitelisted `ORDER BY`, the sort rule holds in Arms 2–5, 10, and 11, and the cross-arm contamination from round 2 is gone. But seven problems block registration:

1. **Arms 7 and 9 still fail hydration** in form, dashboard, and browser at every payload size (30 of 30 such pages). H10's client-equivalence premise is false as built.
2. **The ablation still lacks its patterns.** Arm 9 has none of its five patterns; Arm 7's server tree has 8 to 22 hook calls per archetype; Arm 8's has none.
3. **The harness cannot run the study.** Full-run mode is unimplemented, and 55 of 55 interaction scripts fail on selectors.
4. **Arm 6 is not on SQLite** and still sorts with `localeCompare`, yet the new preregistration paragraph says it queries SQLite per interaction.
5. **One bad grid value takes down the whole Node server** (Arms 1–10). Arm 11's row selection does not work in a browser.
6. **The preregistration now states false facts**: byte-identical server HTML across Arms 7–9 and per-interaction SQLite for Arm 6. The power figure is still wrong.
7. **The build is not reproducible.** The checked-in JSON and SQLite files do not match the generator, there are no lockfiles or engine pins, `better-sqlite3` 13 crashes on the Node 20 the spec names, and Arm 11 lacks a dependency it needs.

Status counts across the 57 tracked items (M1–M8, minor 1–14, A-1 to A-23, and N1–N12): 12 fixed, 24 partly fixed, 20 not fixed, and 1 no longer applies. Of the 45 items from rounds 1 and 1a alone: 12 fixed, 23 partly fixed, 9 not fixed, and 1 no longer applies (round 2: 11, 17, 16, and 1).

Status of earlier issues

Round 1 major issues

ID	Issue	Status	Evidence
M1	Ablation arms lack their patterns	Partly fixed	Unchanged from round 2. Arm 8 has per-row classes. Arm 9's server trees have no `createContext` or provider, no class, no `switch`, no custom hook, and no `children` wrapper. See N3.
M2	Arms do not implement the task spec	Partly fixed	No virtual scrolling, Chart.js, grid subtotal rows, payment step, Luhn check, or detail panel in any arm (`rg -i "virtual|chart.js"` finds nothing in `apps/` source).
M3	Harness cannot run; selectors do not match	Partly fixed	`harness/src/` is unchanged since round 2. `runner.js:340` still prints "Full run mode not yet implemented." Interaction scripts: 0 of 55 arm-archetype pairs complete (see R3-7). No run-order randomization.
M4	Measurement confounds favor SSR arms	Partly fixed	Still no compression on either server (no `Content-Encoding` on any response). `server.js:89` still sends a 302 to port 4001 for Arm 11. New: COOP and COEP are set only by Node (`server.js:101–102`); see R3-12.
M5	SSR arms do less work on table (ID bugs)	Fixed	Verified over HTTP: Arm 10 and Arm 11 edit and commit return 200 and apply.
M6	Conflict-of-interest controls not auditable	Partly fixed	Unchanged. No independent programmer, AI-assistance disclosure, time log, build history, or freeze DOI.
M7	Statistical families and null tests	Partly fixed	H10's TOST now covers INP and TTI (lines 227, 254). The power figure at line 121 is unchanged (N4). H6 and H7 still compare settled time with INP (N5).
M8	Data generation not reproducible	Partly fixed	Verified: the generator run under UTC differs from the checked-in JSON in 15 of 25 files (all of table, dashboard, and browser); 36 of 100 table rows have a `hireDate` one day later in the checked-in file. The SQLite manifest hashes the stale files. No lockfile in `apps/` or `harness/`; no `engines` field. See R3-6.

Round 1 minor issues

ID	Issue	Status	Evidence
1	"Every payload size" for bundle size	Partly fixed	H3 (`preregistration.md:35`) still says it.
2	Old arm name in H6 analysis	Fixed	Still absent.
3	Arm 6 compared with Arm 9	Partly fixed	Line 99 still says "against Arm 9."
4	H12 "same HTML fragments"	Partly fixed	Arm 10 and Arm 11 markup still differ: Arm 11 lacks `data-column`, `data-row`, `data-action`, and `data-role` on the harness targets, and the table shows "1 selected" against Arm 10's "1 of 1000 selected."
5	Blinding; analysis script	Partly fixed	Line 79 still says "Not applicable." No analysis script in the tree.
6	TTFB claim not validated	Not fixed	Line 185 is reworded around the four Server-Timing phases. There is still no fixture that checks the claim.
7	p98 of about 10 interactions is the maximum	Fixed	Line 197 says the 98th percentile "is effectively the maximum." The line did not change in this round; my round-2 status was wrong.
8	`rows` not validated	Fixed	`ALLOWED_SIZES` remains in both servers.
9	EJS caching	Fixed	`{ cache: true }` (e.g., `grid/10-htmx-ejs/server.js:46`).
10	Single worker and CPU sharing unstated	Not fixed	Not mentioned.
11	128 MB of data in git	Not fixed	`data/` is 128 MB; `data/sqlite/` adds untracked database files.

ID	Issue	Status	Evidence
12	VPS details	Not fixed	Line 140 gives CPU, RAM, and OS. Provider, kernel, Chrome version, and frequency scaling are still missing.
13	Mobile profile throttles CPU only	Not fixed	Not stated.
14	No threats-to-validity section	Not fixed	None found.

Addendum items (A-1 to A-23)

ID	Item	Status	Evidence
A-1	One shared client per archetype, from Arm 1	Fixed	`apps/<archetype>/shared-client/App.jsx`.
A-2	Identical `index.jsx` with `hydrateRoot`	Fixed	Unchanged.
A-3	Equal bundle hashes for 7, 8, and 9	Fixed	Verified: `node build.js` built 45 bundles; `dist/app.js` is identical within each archetype for Arms 7–9 (e.g., table `01087b0c…`).
A-4	Client-only patterns removed from 8 and 9	Fixed	Client is shared.
A-5	Server HTML matches the shared client's first render	Partly fixed	Verified in Chrome: Arm 8 clean everywhere; Arms 7 and 9 fail in form, dashboard, and browser. See N1.
A-6	Arm 7 server tree with no hooks	Not fixed	Hook calls in Arm 7 server trees: table 22, form 8, dashboard 16, browser 14, grid 19.
A-7	Arm 8 per-row classes; raw JSON to client	Fixed	Unchanged.
A-8	Arm 9's five patterns in the server tree	Not fixed	None present. Hook calls: table 21, form 9, dashboard 14, browser 12, grid 18.
A-9	Canonical sort in every arm	Partly fixed	Verified on full order at 100,000 rows, every column of `employees`, `orders`, and `browser_records`, both directions: shared client, Arms 2–5, and the Arm 10 and Arm 11 SQL show 0 mismatches. Arm 6 still uses `localeCompare` (`table/6-…/App.jsx:161`, dashboard `:105`, browser `:98`) and misorders 258 email rows.
A-10	`toLocaleString('en-US')`	Partly fixed	Still unpinned in Arms 2–6 and 10 (e.g., `table/2-redux/App.jsx:49`, `grid/10-htmx-ejs/server.js:17`, `table/10-htmx-ejs/views/_content.ejs:88`).
A-11	Integer cents for money	Partly fixed	Stored as integer cents in SQLite. JavaScript divides by 100 on read (`lib/sqlite.js:75–83`). Python sums floats after dividing (`lib/sqlite_db.py:274`); see R3-9.
A-12	One salary parse rule	Partly fixed	JavaScript uses `/^\d+$/` (`lib/sqlite.js:269`). Python uses `str.isdigit()` (`lib/sqlite_db.py:111`), which accepts Unicode digits. Verified: Python saved "١٢" as 12 and returned 500 for "²"; JavaScript rejected both.
A-13	SQLite file, schema, indexes, and manifest	Partly fixed	`data/sqlite/schema.sql`, `build.js`, and `manifest.json` exist. Arm 6 does not use SQLite (R3-3). `build.js:4` hardcodes `/Users/adam/dev/perf/...`.
A-14	`instr` filter, whitelisted `ORDER BY`, `LIMIT`/`OFFSET`	Partly fixed	Done in Arms 10 and 11. Not in Arm 6, which receives the full array (`server.js:21`, `:37–47`).
A-15	Matching driver settings and recorded versions	Partly fixed	Pragmas match (`lib/sqlite.js:6–7`, `lib/sqlite_db.py:8–19`). The manifest records different SQLite versions for the two runtimes (3.53.4 and 3.54.1). Python keeps its default implicit-transaction mode.
A-16	Per-run working copy and reset endpoint	Partly fixed	Each page load makes a temp copy (`mkdtemp` and `copy2`, Arm 11 `routes.py:106–111`; Arm 10 likewise). The copies are never removed (28 `bench-*` directories in `/tmp` after the audit). `reset()` in

ID	Item	Status	Evidence
			`lib/sqlite.js:285–290` is never called and leaves a closed connection (R3-10). There is no reset endpoint.
A-17	Parse IDs at the route boundary	Fixed	IDs parse; bad-input handling is tracked under N12.
A-18	Split Server-Timing	Partly fixed	Node SSR sends `load`, `map`, `render`, and `serialize`. Arms 10 and 11 hardcode `map` and `serialize` to zero (`server.js:104–119`; `fastapi-server.py:72`), include the file copy in `load`, and send no Server-Timing on htmx partials. See R3-5.
A-19	CI checks for bundle hash, markup parity, and hydration	Not fixed	No CI. `apps/parity-check.mjs` covers Arm 8 only, at 1,000 rows, from JSON.
A-20	Preregistration amendments	Partly fixed	A data-access paragraph now exists (line 101) and H10 is restated (lines 49, 185, 227, 252, 254). Two of its claims are false (R3-4).
A-21	Stale spec passages	Partly fixed	`task-specification.md` is unchanged since round 2; line 21 still contradicts line 67.
A-22	Rebuild Arm 6 to spec	No longer applies	Spec v0.2 defines Arm 6 as it is built.
A-23	Same initial data source for 7, 8, and 9	Fixed	All three now read the same SQLite file through `loadDataFromSqlite` (`server.js:37–47`).

Round 2 new issues (N1–N12)

ID	Issue	Status	Evidence
N1	Arms 7 and 9 fail hydration	Not fixed	Verified in Chrome at 100, 1,000, 10,000, 50,000, and 100,000 rows: Arms 7 and 9 throw #418 and #425 and replace the app on form, dashboard, and browser (8, 8, and 28 errors per page). Arms 6 and 8 are clean; table and grid are clean in all arms. Cause in form: the shared client starts empty and prefills in an effect, and renders `{label} {'*'}`; Arm 7 and 9 servers prefill (`value="William"`) and lay out label text differently.
N2	Shared data cache contaminates arms	Partly fixed	Verified: after an Arm 10 commit, the value did not appear in Arms 6, 7, 9, or 11. But `col` is still not whitelisted: Arm 10 edit-cell with `col=__proto__` returns 200 with `value="[object Object]"` (`table/10-htmx-ejs/server.js:136`); Arm 11 returns 500 (`table/11-…/routes.py:202`).
N3	Ablation order inverted	Not fixed	See A-6 and A-8. Arm 8's `ServerApp` has 0 hooks in every archetype.
N4	Power statement wrong	Not fixed	Line 121 unchanged. Recomputed: Cliff's delta 0.45 ≈ d 0.85; at α = 0.0005 two-sided and n = 30, power ≈ 0.4. Eighty percent needs about 55–60 per group.
N5	Settled metric fragile; H6/H7 mix metrics	Not fixed	`web-vitals.js` unchanged (no quiet window). Lines 41, 43, and 246 still compare Arm 10 settled time with CSR INP.
N6	0 ms profile called uncapped	Not fixed	Line 75 caps bandwidth "for all non-zero latency levels"; `profiles.js:6` caps 0 ms too.
N7	Spec contradictions	Not fixed	Spec unchanged. Line 19 also still names Node 20 and "static JSON files."
N8	Broken sentence in research plan	Not fixed	`research-plan.md` unchanged.
N9	Locale split	Not fixed	See A-10.
N10	`__SSR_DATA__` not escaped	Not fixed	`server.js:160`.
N11	Dead code in ablation arms	Not fixed	Arm 8 `App.jsx` imported by nothing; `Intl.Collator('en-US')` still in Arm 7 `lib.js` and Arm 9 `App.jsx` (table:12, dashboard:16, browser:13).

ID	Issue	Status	Evidence
N12	Input validation differs by runtime	Not fixed	Verified: Arm 11 returns 500 for `id=abc` and `p=abc`; Arm 10 `parseInt` reads "1abc" as id 1. See R3-1 and R3-8 for value parsing.

New issues

Major

R3-1. One malformed grid value crashes the Node server for Arms 1–10 (verified). *Location:* `grid/10-htmx-ejs/server.js:125`; `lib/sqlite.js:328` (`updateCell`). *Evidence:* `POST /grid/10-htmx-ejs/commit-edit` with `editValue=Infinity`, and separately `bulk-fill` with `bulkValue=Infinity`, each ended the Node process. The log shows `SqliteError: cannot store REAL value in INTEGER column budget_lines.q1` (`SQLITE_CONSTRAINT_DATATYPE`), uncaught in an async Express handler. Every Node-served arm was then down until restart. `parseFloat` also stores "12abc" as 12. *Fix:* parse with a strict decimal pattern, reject non-finite values, round to integer cents before the write, wrap handler bodies in `try`/`catch` with an Express error handler, and add a test for these inputs.

R3-2. Arm 11 row selection does not work in a browser (verified). *Location:* `grid/11-fastapi-honest-htmx/templates/_content.html:14, 38, 53` (and the table template); grid `routes.py` toggle handlers. *Evidence:* `hx-vals` embeds `selected_json` inside a JSON string. Jinja escapes its quotes to `"`, the browser decodes them, and htmx gets invalid JSON (`SyntaxError: Expected ',' or '}' … at position 64`). In Chrome, after clicking two row checkboxes, Arm 11 table and grid each show one checked row; Arm 10 shows two and deselects correctly. Separately, `toggle_row` and `toggle_all_visible` use string keys while `_parse_state` converts keys to `int`, so deselection and counts can break even with valid input (from code reading). *Fix:* pass selection as a hidden input or serialize `hx-vals` with `|tojson`; use one key type; add a browser test that selects, deselects, and selects all.

R3-3. Arm 6 is not on SQLite and breaks the sort rule. *Location:* `apps/server.js:21` (Arm 6 in `SSR_ARMS`), `:37–47`; `table/6-…/App.jsx:161`. *Evidence:* Arm 6 receives the full dataset like Arms 7–9 and never pages from the database. Its `localeCompare` misorders 258 email rows at 100,000 rows, in both directions. *Fix:* route Arm 6's interactions through the SQLite page query, as the plan says, or change the plan; replace `localeCompare` with the shared ASCII comparator.

R3-4. The amended preregistration states things that are false. *Location:* `preregistration.md:49` and `:101`. *Evidence:* Line 49 says Arms 7–9 "server HTML is byte-identical, verified per archetype and payload size." It is not, in form, dashboard, and browser (N1). Line 101 says Arms 6, 10, and 11 "query a SQLite file per interaction"; Arm 6 does not (R3-3). *Fix:* make the code true to the text, then add the check that verifies it (A-19), or correct the text.

R3-5. Server-Timing is not comparable across arms. *Location:* `apps/server.js:104–119`; `fastapi-server.py:72`; Arm 11 `routes.py:106–111`. *Evidence:* The htmx arms report `map` and `serialize` as fixed zeros. `load` includes a full database file copy on every page load. htmx partial responses carry no Server-Timing at all, so H12's interaction-level comparison has no data source. *Fix:* time the same phases in Node and Python, move the working copy outside the timed path (once per run), and emit Server-Timing on partials.

R3-6. The build cannot be reproduced from the repository. *Location:* `data/`, `data/sqlite/build.js:4`, `apps/package.json`, `apps/pyproject.toml`, `apps/uv.lock`, spec line 19. *Evidence:* (a) Checked-in JSON differs from fresh generator output in 15 of 25 files (off-by-one-day dates), and the SQLite files are built from those stale files. (b) `better-sqlite3` 13 segfaults on Node 20, the version the spec names; there is no `engines` field and no `package-lock.json` (excluded by `.gitignore`). (c) `python-multipart` is missing from `pyproject.toml` and `uv.lock`, but `request.form()` requires it, so Arm 11's POST routes fail on a clean install. (d) `build.js` hardcodes the author's

home path. *Fix:* regenerate and commit data, rebuild the SQLite files and manifest, pin Node 22 in `engines`, commit lockfiles, add `python-multipart`, and use relative paths.

R3-7. Every harness interaction script fails (verified). *Location:* harness interaction scripts; Arm 10 and Arm 11 templates; shared client. *Evidence:* In correctness-only runs at 1,000 rows, 0 of 55 arm-archetype pairs complete. Failing selectors: form `[name="firstName"]` (client uses `data-field`); dashboard `[name="startDate"]`; browser `[data-role="virtual-list"]`; grid `tr[data-row="0"] td[data-column="q1"]`; table `[data-action="next-page"]` (Arms 1–9), `td[data-column="name"]:first-of-type` (Arm 10), and `th[data-column="name"]` (Arm 11). *Fix:* define one selector contract per archetype in the spec, apply it to all 11 arms, and require a clean pass on all 55 pairs before registration.

Minor

- **R3-8. Value parsing differs by runtime.** Salary: Python accepts Unicode digits (`sqlite_db.py:111`; "١٢" saved as 12, "²" gives 500). Grid: JavaScript accepts "12abc" as 12; Python rejects it, and "inf" raises an uncaught `OverflowError` (500) at `sqlite_db.py:289/303`. Both accept "1e3." Fix: one written rule, applied in both runtimes and tested with the same input list.
- **R3-9. Python grid totals use float arithmetic.** `sqlite_db.py:274` divides before summing. Arm 11 pages carry values such as `100987.54000000004` (41 to 45 per page at 1,000 and 100,000 rows); Arm 10 sums cents. Fix: sum integers, divide only to format.
- **R3-10. `lib/sqlite.js` has dead and unsafe paths.** `reset()` (lines 285–290) is never called; after it, `update`, `getById`, and `delete` throw "The database connection is not open." `updateStmt` (line 263) is unused. `update()` returns `true` for unknown columns and missing rows. Statements are prepared per call.
- **R3-11. The suggested shared collator conflicts with the sort rule.** `new Intl.Collator(undefined, {sensitivity: 'base'}).compare` misorders the same 258 email rows as `localeCompare` (first at position 4,200 ascending and 1,348 descending, at 100,000 rows); all other columns match. It is also locale-dependent. Keep the ASCII `toLowerCase` comparator with the `id` tie-break, which matches SQLite `NOCASE` on every column.
- **R3-12. COOP and COEP headers differ by server.** Node sets them (`server.js:101–102`); FastAPI does not. Cross-origin isolation changes timer resolution and some browser behavior. Set the same headers on both or neither.
- **R3-13. Some Python queries sort without `NOCASE`.** `dashboard_categories` and `dashboard_statuses` (`sqlite_db.py:189, 193`). Harmless on current data, but align with the rule.
- **R3-14. Tree hygiene.** 28 parity scratch files (`apps/arm8-*.mjs`, `apps/shared-*.mjs`) and 50 macOS `._*` files are untracked; `vendor/htmx` and `data/sqlite` are untracked; a copy of our round-2 review sits in the tree root. Clean these before the freeze deposit.

Must fix before registration (in priority order)

1. **Hydration parity (N1, A-5).** Rewrite the Arm 7 and 9 server trees for form, dashboard, and browser to match the shared client's first render, and add a Chrome check that fails on any hydration error at all five sizes.
2. **Ablation content (N3, A-6, A-8, M1).** Make Arm 7 hook-free, build Arm 9's five patterns, and remove dead code (N11).
3. **Harness (M3, R3-7, N5).** Implement full-run mode and run-order randomization, fix the selectors, and add a quiet window to the settled metric. Then do a pilot that passes all 55 pairs.
4. **Server robustness (R3-1, R3-2, N2, N12, R3-8).** Stop the crash, fix Arm 11 selection, whitelist columns, and use one validation rule in both runtimes.
5. **Arm 6 data path (R3-3, A-13, A-14).** Move it to SQLite paging and the ASCII comparator, or change the plan.

6. **Measurement parity (R3-5, M4, R3-12, A-16).** Comparable Server-Timing phases, partials timed, working copy outside the timed path, compression decided for both servers, no 302, and matching headers.
7. **Reproducibility (R3-6, M8, A-15).** Regenerated data, rebuilt SQLite, lockfiles, Node pin, and complete Python dependencies.
8. **Text (R3-4, N4, N6, N7, N8, minor 1, 3, 5, 6, 10–14).** Correct the false claims and the power figure, align the spec, and add the threats-to-validity and environment details.

Verification results

Check	Result
Bundle build and identity	45 of 45 built; Arms 7–9 identical within each archetype
Hydration in Chrome, Arms 6–9, five archetypes, five sizes	Arms 6 and 8 clean; Arms 7 and 9 fail 30 of 30 form, dashboard, and browser pages
Full-order sort at 100,000 rows, all columns, both directions	Shared client, Arms 2–5, and Arm 10 and 11 SQL: 0 mismatches; `localeCompare` and `Intl.Collator`: 258 email rows
Cross-arm state after an Arm 10 edit	Not visible in Arms 6, 7, 9, or 11
Arm 10 and 11 input handling	See N2, N12, R3-1, and R3-8
Arm 11 row selection in Chrome	Fails (one row kept of two); Arm 10 passes
Harness interaction scripts	0 of 55 complete
Generator vs. checked-in data	15 of 25 files differ

What I did not verify

- Any timing. By instruction, no performance measurement was made or read.
- Conditions on the study VPS.
- Arm 11 through the Node 302 redirect; I tested it on its own port.
- Hydration of Arms 1–5 (they use `createRoot`) and of Arms 10 and 11 (no React).